

Making a Rewarding RL Project

Mateus Karvat Camara

MAY 01, 2026

Who is Mateus?

- Well, the TA
 - Email: mateus.k@queensu.ca
- PhD student under Dr. Givigi
- Took CISC 856 in 2023
- Worked on 2 Multi-agent Reinforcement Learning (MARL) research projects with Dr. Givigi
- Worked as Teaching Assistant for RL courses three times

What is the project about?

What is the project about?

- Well, it's open-ended
- Possible approaches:
 - Replicating and contrasting existing studies (Benchmarking)
 - Expanding a published paper/study (Paper expansion)
 - Modeling a novel problem and solving it (Novel problem – talk to us about it)

What we expect (no matter the type of project)

- Understand the algorithms being used
 - Especially if they are not traditional
- Look into similar approaches
- Comprehensive experimentation
 - Not just 2 or 3 runs, but several different experimental conditions
 - **Avoid problems/algorithms that would take way too long to train (MARL, Vision, training LLM/VLM/VLA)**
- Depth of analysis
 - Tables, plots, relevant metrics
 - Why is X better than Y?

What we expect (no matter the type of project)

- Project report = first version of a paper
 - Organization
 - Follow IEEE conference paper template
 - Include mathematical descriptions of the Action space, the Observation space, and the Reward functions in your Methodology!
 - Rigor
 - Use citations to support your claims
 - As much as possible, try to quantify results/methods
 - Discuss your results instead of just mentioning them

What we expect (no matter the type of project)

- **Finish the project!**
 - Environment + Algorithms
 - The task does not need to be successfully completed, but we need to see significant progress and attempts at solving it
 - Use existing tools
 - Gymnasium
 - **Stable Baselines 3 (don't code your RL algorithm from scratch)**
 - Pygame (if relevant)
 - Tensorboard or equivalent (for learning plots)
 - **Don't be overly ambitious!**

Benchmarking

- Read a few papers that deal with the same problem
- Make sure to understand their approach
- Papers that have open source implementations are best
 - Especially if they have weights on Hugging Face
- Run them all, compare their results and maybe test them in novel scenarios

Benchmarking - example

Abstract—In this paper, we conduct a reinforcement learning case study on Atari 2600's game *Gopher*. *Gopher* is a discrete-spaced, fully observable video game in which a farmer protects three carrots from a hungry gopher. Due to the challenging and fast-paced nature of the game, it is a compelling domain to compare the scores achieved by trained reinforcement learning agents with human scores; and while *Gopher* has been included in past Atari reinforcement learning experiments, *Gopher* alone has not been extensively studied. Our project conducts an extensive and systematic analysis of 5 state-of-the-art algorithms—DQN, QR-DQN, A2C, PPO, and RecurrentPPO—with the goal of studying how their performance varies with different environment settings when applied to *Gopher*.

Index Terms—DQN, PPO, A2C, QR-DQN, R-PPO, Reinforcement Learning, Atari 2600, *Gopher*

I. INTRODUCTION

The Atari 2600 suite of games is frequently used to experiment with and study the behaviours of various reinforcement learning (RL) algorithms. The Gymnasium [1] and Arcade Learning Environment (ALE) [2] have provided a standard benchmark for a large selection of Atari games, and as such, new algorithms are tested on a wide variety of Atari games at once. One of the games frequently included in these large-scale studies is *Gopher*.

Gopher (1982) is a single-screen action video game entry in the Atari 2600 library of games (Figure 1). In the game, a player controls a farmer wielding a shovel protecting a crop of three carrots from a digging, gluttonous, conniving gopher¹. They use a joystick to move and a button to fill tunnels, “bonk” the gopher, or plant seeds dropped by a duck, with the objective of the game being to prevent the gopher from tunneling to the surface and eating all three carrots.

Early agents such as DQN treated *Gopher* as one of many evaluation domains, reporting strong performance alongside other Atari games [3] [4]. More recent work has shifted toward sample-efficient learning, with agents trained on as little as 100k frames (approximately two hours of gameplay). For example, IRIS [5], a transformer-based world-model agent, significantly outperformed prior sample-efficient baselines such as SimPLe [6], CURL [7], DrQ [8], and SPR [9] across several Atari tasks, including *Gopher*. Other work

has been done with *Gopher* to test more advanced techniques, such as *Causal State Distillation* [10], which uses *Gopher*'s multi-reward structure to learn causal explanations.

The work done in this project does not focus on benchmarking top scores in *Gopher*, but rather studying how different environment settings affect the behavioural and learning dynamics of various RL algorithms when applied to the game.

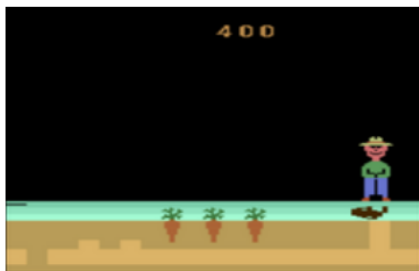


Fig. 1. Screenshot of *Gopher* Atari 2600 gameplay.

II. PROBLEM FORMULATION

This project focuses on examining the performance of various RL methods when applied to the Atari *Gopher* environment. As noted in an extensive study comparing *Gopher* scores across various algorithms [11], the most recent standardized comparison dates back to 2018. Therefore, the motivation behind this work is to investigate and evaluate the latest advancements in RL to assess how modern approaches compare to prior results.

Several algorithms with different attributes were chosen to compare alongside DQN, the current highest-performing RL algorithm on Atari games. These are: PPO, R-PPO, QR-DQN, and A2C.

III. ENVIRONMENT

In *Gopher*, the player takes control of a farmer who is protecting three carrots from being eaten by a gopher. The gopher travels underground beneath the farmer and digs tunnels to

- Problem: *Gopher*
 - ATARI game with existing Gymnasium environment
- 5 algorithms compared:
 - DQN
 - PPO
 - A2C
 - QR-DQN
 - R-PPO

¹Gopher: <https://en.wikipedia.org/wiki/Gopher>

Benchmarking - example

TABLE I
TRAINING CONFIGURATIONS

Variation	Rewards ($\mathcal{R}$)	Frameskip (F)	Learning Rate (α)	Description of Training
Version 1 (V1)	Clipped: $\{-1,0,+1\}$	Static: 4	Default (respectively)	Default StableBaseline model settings, used as the baseline configuration to for comparing impact of reward shaping, frameskip changes, and learning-rate adjustments.
Version 2 (V2)	Actual: $\{\text{bonk}=+100, \text{fill}=+20\}$	Static: 4	Default (respectively)	Custom reward function (actual reward scoring of the game) to test whether denser and more task-specific rewards could better guide exploration and encourage behaviors more aligned with game's objectives.
Version 3 (V3)	Clipped: $\{-1,0,+1\}$	Stochastic: $[4,10]$	Default (respectively)	Replaced fixed frameskip with wider stochastic range to examine whether increased temporal variability would improve robustness and reduce overfitting to a single action repetition pattern.
Version 4 (V4)	Clipped: $\{-1,0,+1\}$	Stochastic: $[3,5]$	Default (respectively)	Used a narrower stochastic frameskip range to test whether smaller amount of action-timing randomness could preserve stability while still providing same benefits hypothesized in V4.
Version 5 (V5)	Clipped: $\{-1,0,+1\}$	Static: 4	0.01	(Experimental) Designed to evaluate how a substantially larger learning rate affects training speed and stability, and whether faster parameter updates help or destabilize performance.

TABLE II
MEAN REWARD OVER 1000 EPISODES — V1 (CLIPPED) AND V2 (ACTUAL REWARDS)

Model	V1: Clipped		V2: Actual	
	Best	Final	Best	Final
DQN	1110.30	1067.94	813.42	700.86
QR-DQN	414.78	342.52	277.42	313.78
A2C	667.06	396.46	468.34	—
PPO	176.02	103.18	57.44	27.06
R-PPO	411.70	586.84	355.84	—
Human	2412.5			

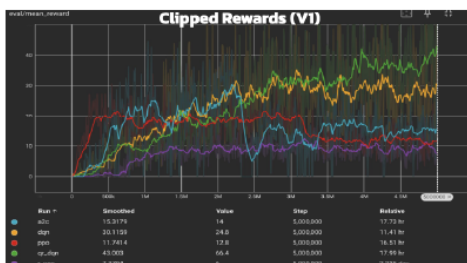


Fig. 2. Training curves (eval/mean_reward) for all agents under V1 (clipped rewards, frameskip=4) over 5,000,000 timesteps.

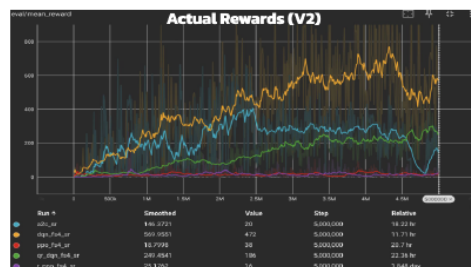


Fig. 3. Training curves (eval/mean_reward) for all agents under V2 (actual rewards, frameskip=4) over 5,000,000 timesteps.

variation. QR-DQN improved to 449.84 relative to V3. A2C achieved a best-checkpoint score of 290.20 but its final checkpoint declined to 102.88, indicating policy instability late in training. PPO remained the weakest performer (74.98 best). These results indicate that *Gopher*'s fast-paced, reactive gameplay makes performance sensitive to frameskip: wider frameskip ranges (V3) introduce lag that disrupts time-critical actions such as filling tunnels and bonking the gopher, harming most agents. The training curves for V4 are shown in Fig. 5.

- Extensive experimentation
 - Unique training configurations proposed (Table I)
 - Tables presenting final results (Table II)
 - Plots showing the results throughout training (Fig 2, Fig 3)

Paper Expansion

- Pick a paper and make sure to DEEPLY understand its approach
- Look for gaps and issues in their methodology
 - Techniques they could have used
 - Scenarios they didn't explore
 - Subpar results
 - Hint: Future work section
- Replicate their experiments
 - If you get different results, investigate why
- Implement your improvement ideas and conduct comprehensive experiments

Paper Expansion - example

III. METHODOLOGY

Our proposed agent architecture follows the methodology established by Xu et al. [1]. The agent operates as a learning rate (LR) controller that observes a state vector of training metrics from the environment every k optimizer steps. Based on these observations, the agent outputs an action value that scales the current LR, adaptively navigating the loss landscape. A high-level overview of the agent and environment is illustrated in Figure 1.

A. *Soft Actor-Critic*

While Xu et al. utilize Proximal Policy Optimization (PPO) to learn the controller policy, we propose Soft Actor-Critic (SAC) as an alternative [7]. SAC provides three main advantages over PPO: (1) its off-policy architecture uses a replay buffer to learn from past experience, significantly improving sample efficiency, (2) its maximum entropy objective prevents premature convergence by improving exploration, and (3) it is inherently less sensitive to its own initial hyperparameters. Additionally, modern implementations of SAC dynamically adjust the temperature parameter α to maintain a target entropy. This eliminates the need to manually tune the exploration-exploitation trade-off, which is especially advantageous when the optimal value may differ between different training environments.

- Paper selected:
 - Xu et al, 2019 - Learning an Adaptive Learning Rate Schedule
- Gap in the methodology:
 - Original paper used PPO, although SAC is a better alternative

Paper Expansion - example

TABLE I: Test accuracy (%) across model/dataset combinations and observation space dimensions. Results reported as mean $\pm$ std over 3 runs. Best result per row is **bolded**.

Model	Dataset	Fixed LR		Step Decay		Cosine		SAC (Ours)	
		7D	16D	7D	16D	7D	16D	7D	16D
LeNet-5	FashionMNIST	87.53 $\pm$ 0.08	87.54 $\pm$ 0.22	89.69 $\pm$ 0.50	89.50 $\pm$ 0.43	86.41 $\pm$ 0.09	86.42 $\pm$ 0.16	88.93 $\pm$ 0.45	89.81 $\pm$ 0.31
	CIFAR-10	56.27 $\pm$ 0.72	56.33 $\pm$ 0.55	58.41 $\pm$ 2.79	58.31 $\pm$ 3.69	53.76 $\pm$ 1.24	53.71 $\pm$ 1.21	44.80 $\pm$ 3.55	52.44 $\pm$ 1.94
	CIFAR-100	22.99 $\pm$ 1.03	23.18 $\pm$ 0.73	19.35 $\pm$ 2.58	17.87 $\pm$ 4.74	20.17 $\pm$ 1.22	20.09 $\pm$ 1.31	14.78 $\pm$ 7.37	23.08 $\pm$ 4.02
ResNet-18	FashionMNIST	90.81 $\pm$ 0.35	90.34 $\pm$ 0.80	91.98 $\pm$ 0.75	89.63 $\pm$ 0.22	93.65 $\pm$ 0.19	93.84 $\pm$ 0.15	91.68 $\pm$ 1.00	90.12 $\pm$ 0.61
	CIFAR-10	76.96 $\pm$ 3.56	74.66 $\pm$ 4.36	77.04 $\pm$ 1.56	79.86 $\pm$ 2.35	85.84 $\pm$ 0.19	85.68 $\pm$ 0.29	68.66 $\pm$ 0.96	72.69 $\pm$ 2.43
	CIFAR-100	46.41 $\pm$ 2.61	46.55 $\pm$ 2.01	50.00 $\pm$ 1.91	44.30 $\pm$ 2.09	59.64 $\pm$ 0.32	59.60 $\pm$ 0.36	20.28 $\pm$ 1.12	26.85 $\pm$ 6.17

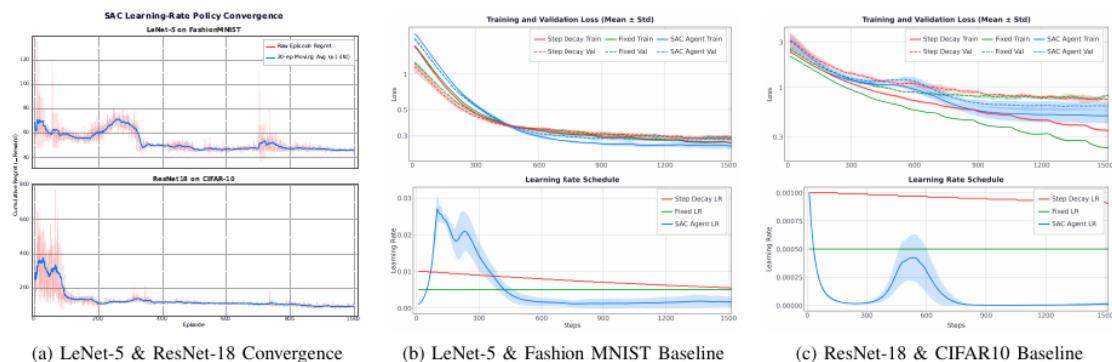


Fig. 2: Policy convergence and Agent vs. Baseline Learning Rate Scheduler evaluation results.

TABLE II: Comparative Performance

Dataset & Model	Scheduler	Test Loss	Test Acc
Fa. MNIST & LeNet-5	Fixed LR	0.298 $\pm$ 0.006	89.2% $\pm$ 0.2%
	Step Decay	0.288 $\pm$ 0.011	89.4% $\pm$ 0.4%
	SAC Agent	0.284 $\pm$ 0.016	89.6% $\pm$ 0.7%
ResNet-18 & CIFAR-10	Fixed LR	0.736 $\pm$ 0.043	78.0% $\pm$ 1.3%
	Step Decay	0.650 $\pm$ 0.064	78.8% $\pm$ 1.9%
	SAC Agent	0.628 $\pm$ 0.065	78.3% $\pm$ 2.3%

TABLE III: Action Scale on Performance (LeNet-5 on Fashion MNIST)

TABLE IV: Alternate Reward Policy Performance (LeNet-5 on Fashion MNIST)

Scheduler	Test Loss	Test Acc
SAC Agent	0.3878 $\pm$ 0.0560	85.84% $\pm$ 1.70%
Alternate Reward SAC Agent	0.0202 $\pm$ 0.0096	87.82% $\pm$ 0.78%

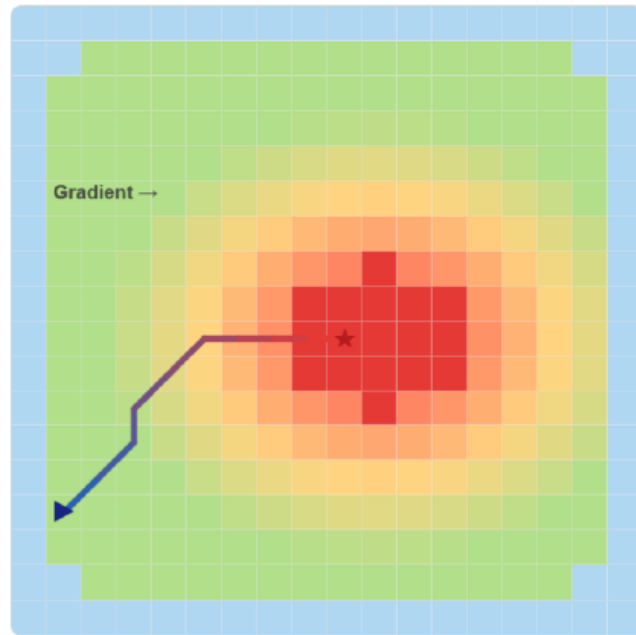
agent demonstrates strong generalization, outperforming both the fixed and step decay baseline schedules by achieving a final test accuracy of 78.89%, compared to 75.85% and 73.32%, respectively.

- Extensive experimentation conducted
 - Students were unable to replicate the results from the original paper (PPO)
 - But they presented the same baseline as the original paper: Step decay
 - They also added 2 new baselines: Fixed RL, Cosine

Novel Problem

- Pick a problem you are already familiar with
 - The schedule for this project is tight, so a completely new problem will be challenging
- Look for current approaches to deal with this problem
 - Look for Gymnasium environments that might fit your problem
- Propose a way to deal with the problem using RL
- Solve the problem iteratively
 - Do a very limited solution to the problem first, considering several constraints
 - Then build from there, adding complexity at every new iteration

Novel Problem - example



Normal tissue forms a **spatial gradient** around tumor.
Agent follows the gradient toward tumor (curved path).

Fig. 1. The Peri-Tumoural Gradient Hypothesis. Tumours (red) reshape surrounding tissue, creating a morphological gradient (warm→cool). An RL agent follows the gradient toward the tumour (curved path).

- Problem:
 - Tumor detection in gigapixel images
- Students were already familiar with the problem and the literature from this field
 - They were able to just focus on the RL solution for the problem

Novel Problem - example

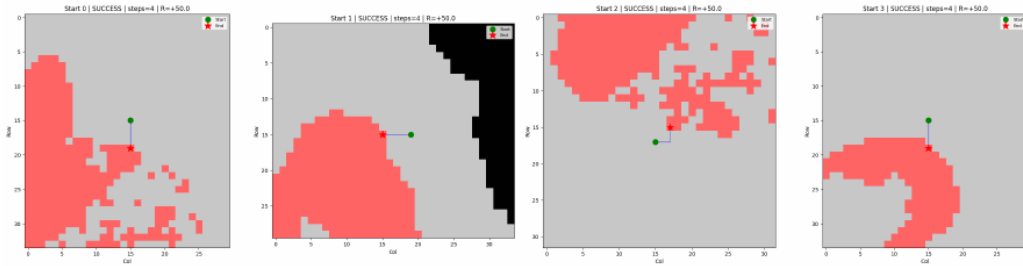


Fig. 3. Experiment 1 trajectories. Four fixed starting positions, all reaching the tumour in 4 steps. Green dot = start, red star = end. Gray = tissue, red = tumour, black = non-tissue.

- Building iteratively
 - They did their experiments on a single gigapixel image
 - There was not enough time to expand their approach for multiple images
 - And that's ok!
 - Their solution worked well on that image
 - They did several experiments on that single image

My CISC 856 Project

Reinforcement Learning Gait Control for Humanoid Robots

Report

- Novel problem
 - Based on my colleague's BSc thesis
 - Center of Mass trajectory for humanoid robots
- Several experiments conducted (over 100)
- Full PDF available as an example here on OnQ
 - This report is being provided just as an example

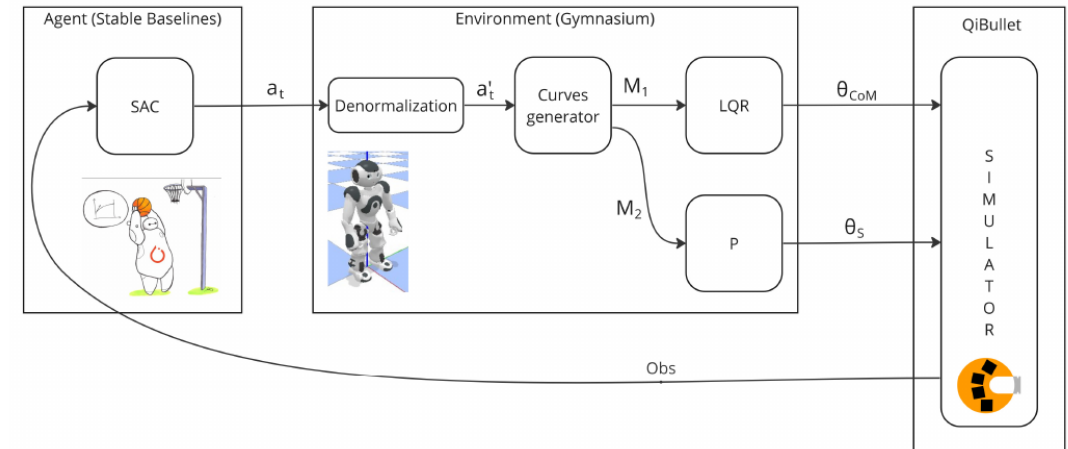
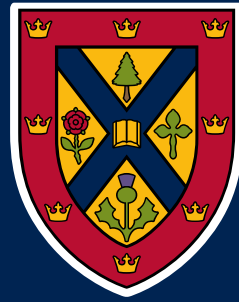


Fig. 1: Pipeline for the proposed implementation. The SAC agent learns parameters for sinusoidal curves that are used by our environment to generate joint angles sent to the simulator. After the simulated NAO robot has performed one step, the simulator returns the observation values. Thus, each step of the robot corresponds to one step of the agent and each training episode runs until the robot falls or until the maximum episode length is reached.



Queen's
UNIVERSITY

Questions:
mateus.k@queensu.ca